

RELATÓRIO TÉCNICO — ViniMarket

Instituição: CESAR School

Disciplina: Sistemas Distribuídos

Aluno: Vinícius Marques

1. Como a Comunicação Entre os Microsserviços Foi Implementada?

A comunicação entre os microsserviços foi implementada por meio do protocolo HTTP/REST (Representational State Transfer), seguindo a arquitetura proposta pela atividade.

O sistema é composto por quatro componentes principais: API Gateway (porta 8000), Serviço de Usuários (porta 5001), Serviço de Produtos (porta 5002 e réplica 5012) e Serviço de Pedidos (porta 5003). O API Gateway atua como único ponto de entrada externo, recebendo todas as requisições do cliente e realizando o encaminhamento (proxy) para o microsserviço correspondente com base no prefixo da rota (/users, /products, /orders).

Toda comunicação interna é realizada com o protocolo HTTP utilizando a biblioteca httpx (Python), que oferece suporte a chamadas assíncronas — essencial para que o gateway não bloqueie enquanto aguarda respostas dos serviços internos. O token JWT é preservado e repassado automaticamente nos cabeçalhos Authorization de cada requisição encaminhada.

Adicionalmente, a comunicação exposta ao exterior (cliente ↔ gateway) foi protegida com TLS/HTTPS por meio de um proxy nginx configurado com certificado auto-assinado (porta 8443), garantindo confidencialidade e integridade dos dados em trânsito.

2. Qual Estratégia de Consistência Foi Adotada na Replicação? Forte ou Eventual? Por Quê?

Foi adotada a estratégia de consistência forte (strong consistency).

Na implementação, toda operação de escrita no Serviço de Produtos (instância primária, porta 5002) é imediatamente propagada para a réplica (porta 5012) por meio do endpoint interno POST /internal/products, de forma síncrona — ou seja, o sucesso ao cliente só é confirmado após ambas as instâncias registrarem o dado. As leituras são distribuídas em round-robin entre as duas réplicas.

Essa escolha foi feita porque, em um sistema de e-commerce, inconsistências nos dados de produtos — como um produto aparecer com preço diferente ou estar disponível em uma réplica mas não em outra — resultariam em experiências ruins ao usuário e possíveis erros de pedido. A consistência forte garante que qualquer leitura, independentemente de qual réplica seja consultada, retornará os dados mais recentes. A desvantagem é a latência adicional nas escritas, que é aceitável dado o volume reduzido deste sistema.

3. O Que Acontece com o Sistema se o Serviço de Pedidos Cair?

Se o Serviço de Pedidos (porta 5003) ficar indisponível, o comportamento do sistema é o seguinte:

- O API Gateway detecta a falha por meio do mecanismo de heartbeat: a cada 5 segundos, o gateway envia GET /health a cada serviço. Após 2 tentativas sem resposta (~10 segundos), o serviço é marcado como indisponível e a falha é registrada em log com timestamp.
- Qualquer requisição a /orders receberá resposta 503 Service Unavailable enquanto o serviço estiver fora.
- Os Serviços de Usuários e Produtos continuam funcionando normalmente, pois são completamente independentes entre si e do Serviço de Pedidos.
- Quando o Serviço de Pedidos voltar a responder ao heartbeat, o gateway registra a recuperação em log e retoma o encaminhamento das requisições automaticamente.

Portanto, o restante do sistema continua funcionando normalmente — o usuário ainda consegue navegar pelos produtos, fazer login e se cadastrar. Apenas as operações de pedido ficam temporariamente indisponíveis.

4. Como o JWT Garante que um Usuário Comum Não Consiga Criar Produtos?

O JWT (JSON Web Token) garante o controle de acesso por meio do campo role incluso no payload do token durante o processo de autenticação. O fluxo de segurança funciona da seguinte forma:

- Emissão do token: no login, o Serviço de Usuários gera um JWT assinado com chave secreta (HMAC-SHA256), contendo userId, email, role ("user" ou "admin") e exp (expiração em 24 horas).
- Envio nas requisições: o cliente inclui o token no cabeçalho Authorization: Bearer <token> em toda requisição protegida.
- Validação no gateway: o gateway repassa o cabeçalho intacto ao serviço de destino.
- Verificação no serviço: o Serviço de Produtos, ao receber POST /products, decodifica e verifica a assinatura do token usando a mesma chave secreta compartilhada. Em seguida, verifica se role == "admin". Caso contrário, retorna 403 Forbidden.

Como o token é assinado digitalmente, qualquer tentativa de alteração do campo role sem conhecer a chave secreta invalida a assinatura, e o token é rejeitado com 401 Unauthorized. Senhas são armazenadas com hash bcrypt (fator de custo adaptativo), garantindo que mesmo em caso de vazamento do banco de dados as senhas não sejam recuperadas em texto plano.

5. Quais Limitações a Implementação Possui em Relação a um Sistema Real de Produção?

A implementação apresenta diversas limitações em comparação com um sistema de e-commerce em produção:

- Banco de dados SQLite: não projetado para alta concorrência com múltiplos escritores simultâneos. Em produção, seriam utilizados PostgreSQL ou MySQL com suporte a transações ACID e conexões paralelas.
- Replicação manual e sem failover automático: não há eleição automática de líder em caso de queda do primário. Em produção, soluções como PostgreSQL Streaming Replication ou sistemas baseados em Raft (ex: etcd) gerenciam failover automaticamente.
- API Gateway como ponto único de falha: se o gateway cair, todo o sistema fica inacessível. Em produção, seria replicado com load balancer e health checks em múltiplas instâncias.
- Certificado TLS auto-assinado: não reconhecido por nenhuma autoridade certificadora (CA), exigindo que o cliente ignore a verificação. Em produção, são utilizados certificados de CAs reconhecidas (Let's Encrypt, DigiCert etc.).
- Ausência de rate limiting e circuit breaker: não há proteção contra abuso de requisições nem mecanismo de circuit breaker para evitar sobrecarga em cascata entre serviços.
- Segredo JWT hardcoded como fallback: a chave secreta padrão presente no código é um risco de segurança se não for substituída por variável de ambiente em todos os ambientes de implantação.
- Ausência de observabilidade: em produção, seriam necessários sistemas de rastreamento distribuído (ex: Jaeger, Zipkin), métricas (Prometheus/Grafana) e centralização de logs (ELK Stack), que vão além do simples log em texto implementado.